University of East Anglia
COMPUTING SCIENCES

Advanced Web Development

Excise 3.1: Next.js dynamic routes and RTK implementation

In this lab, we will implement Next.js dynamic routes and explore RTK for managing global state.

In our previous exercises, we implemented file-system routing in Next.js, where routes strictly follow the directory structure. In this lab exercise, we will expand on that foundation by building dynamic routes, a feature particularly useful in RESTful architectures where pages need to render data based on variable parameters, such as a unique id or slug.

Previously, we also used React's `useState` hook. While `useState` is essential for managing local state within a specific component tree, real-world applications often require a mechanism to share and synchronise state across many independent components.

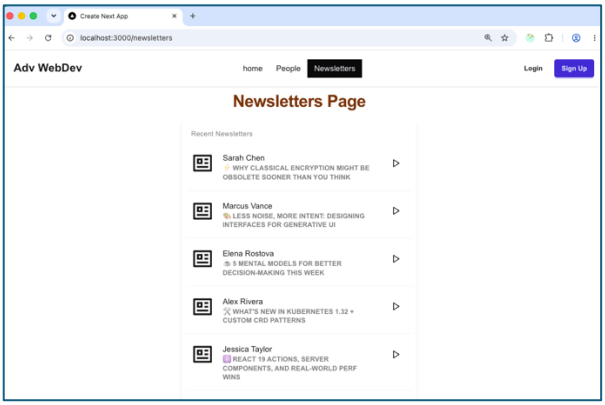
To address this, developers often turn to React's built-in `useContext` hook. Designed for dependency injection, `useContext` allows you to pass data (such as UI themes or user profiles) straight down to deeply nested components without manual prop-drilling. However, it lacks an internal mechanism to optimise performance; any change to a context value forces all consuming components to re-render, which can lead to performance bottlenecks at scale.

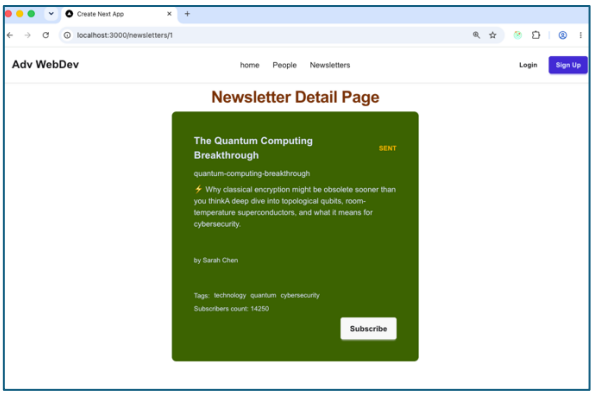
In contrast, Redux Toolkit (RTK) is a comprehensive, production-ready global state management engine. RTK solves the re-render issue by using highly optimised "selectors," ensuring components only re-render when the exact data they depend on changes. Furthermore, it handles complex asynchronous logic out of the box and provides powerful debugging capabilities via Redux DevTools.

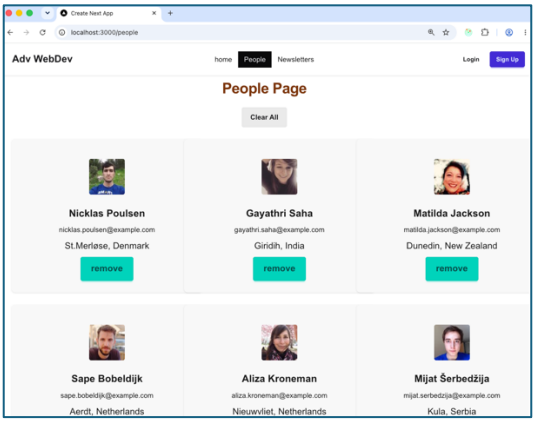
In this lab, we will integrate RTK to centralise application state, streamline data flow, and avoid the pitfalls of prop-drilling. By the end of this exercise, you will be able to handle complex, data-driven navigation and maintain a scalable global state in a modern Next.js application.

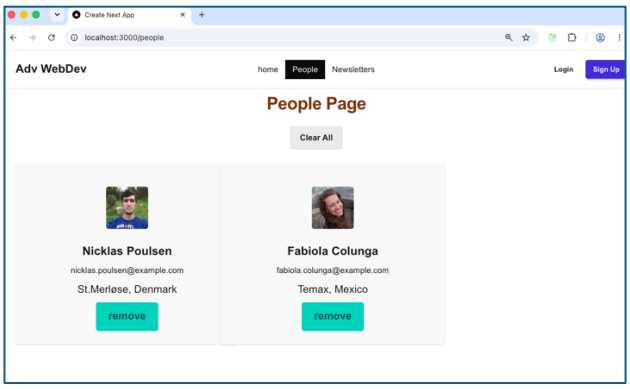
To follow this lab, you must complete Excise 3.0 Next.js Basics and Axios API first.

Examples of final outputs:











Learning Objectives:

- Able to implement dynamic routes on Next.js
- Able to handle error should it occur
- Able to implement Redux Toolkit (RTK) to efficiently fetch, store, and manage global states such as random user data and isLoading.
- Able to apply daisyUI CSS

Tasks:

1. Create a new folder called `lab3.1`. Clone last lab code (lab3) to this folder.
2. Open this folder on VScode
3. Inside the 'newsletters' folder, create:
    a. a new file called `data.json`
    b. a new file called `datatype.ts`
    c. a new folder called `[id]`



4. Next, download the `newsletterData.json` from Blackboard, and copy the data and paste it to this "`data.json`" file.
5. Open the `datatype.ts`, and type the Newsletter dataset, as follow:

app > newsletters > TS dataType.ts > ...

```ts
export interface Author {
  name: string;
  email: string;
}

export interface Stats {
  open_rate: number;
  click_rate: number;
}

export interface Newsletter {
  id: string;
  title: string;
  slug: string;
  author: Author;
  subject_line: string;
  preview_text: string;
  status: "sent" | "scheduled" | "draft";
  sent_at: string | null;
  subscriber_count: number;
  stats: Stats;
  tags: string[];
}
```

6. Open the `page.tsx` in newsletter folder:
    a. Make this a client component
    b. Import the json data

```
3   import rawData from "./data.json";
```

    c. Import the datatype

```
4   import { Author, Stats, Newsletter } from "./dataType";
```

    d. Import Link from next/link

```
6   import Link from "next/link";
```

    e. Import React news icon

```
5   import { RiNewsLine } from "react-icons/ri";
```

    If you have not installed react icons, stop the server, and install the icons: `npm install react-icons --save` and restart the server

    f. Save the rawData (line 9):

```
8   const NewslettersPage = () => {
9     const data = rawData as Newsletter[];
```

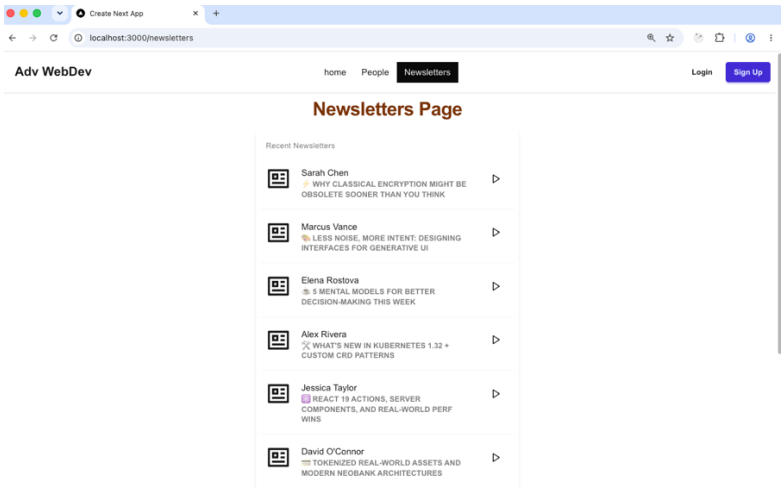
    g. The component will render the newsletter data one by one on the page. For each newsletter,
        i. use React icon

```
24  <div>
25    <RiNewsLine className="size-10 rounded-box" />
26  </div>
```

        ii. there will be a button links to the detail page (of the newsletter). Use “Link” to create the link – to /newsletters/{the id of the newsletter}, see below:

```
<Link key={n.id} href={`/newsletters/${n.id}`}>
```

h. Style the list with daisyUI. Below is my output for your information:



i. Open the `page.tsx` inside the `[id]` folder. This component is server component where we can code async functions.
   i. Import `notFound` from `next/navigation`

```
1   import { notFound } from "next/navigation";
```

   ii. Import the json data

```
2   import rawData from "../data.json";
```

   iii. Type the parameters

```
4   interface PageProps {
5     params: Promise<{ id: string }>;
6   }
```

   iv. The async component will retrieve the `id` from the parameter, then search the record:

```
8   const NewsletterDetailPage = async ({ params }: PageProps) => {
9     const { id } = await params;
10    const newsletter = rawData.find((n) => n.id === id);
11
```

v. In return statement, it will check if record exists, if so, render it, with a subscribe button, otherwise execute notFound, style it with daisyUI. Below is my code:

```
return (
  <>
    {newsletter ? (
      <>
        {" "}
        <h2 className="font-bold text-center text-amber-900 tracking-tight mt-2 text-3xl">
          Newsletter Detail Page
        </h2>
        <div className="p-4 max-w-md mx-auto flex flex-col gap-4 mt-4 card bg-lime-800 text-p
          <div className="card-body">
            <h2 className="card-title">
              {newsletter.title}{" "}
              <span className="text-xs text-amber-400 uppercase">
                {newsletter.status}
              </span>
            </h2>
            <h3>{newsletter.slug}</h3>

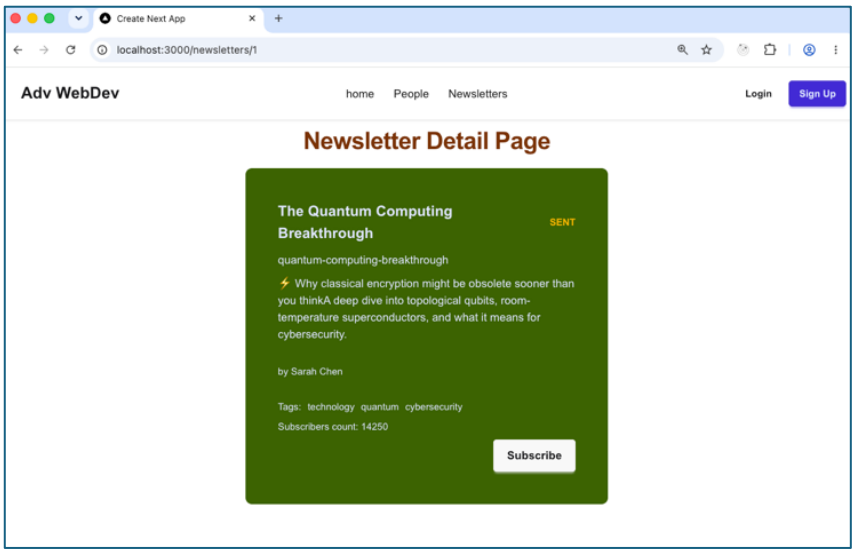
            <p>
              {newsletter.subject_line}
              {newsletter.preview_text}
            </p>

            <div className="text-xs">
              <p className="mt-5">by {newsletter.author.name}</p>
              <div className="mt-5">
                Tags:
                <ul className="menu menu-horizontal gap-2 text-xs">
                  {newsletter.tags.map((t) => {
                    return <li key={t}>{t}</li>;
                  })}
                </ul>
                <p>Subscribers count: {newsletter.subscriber_count}</p>
              </div>
            </div>

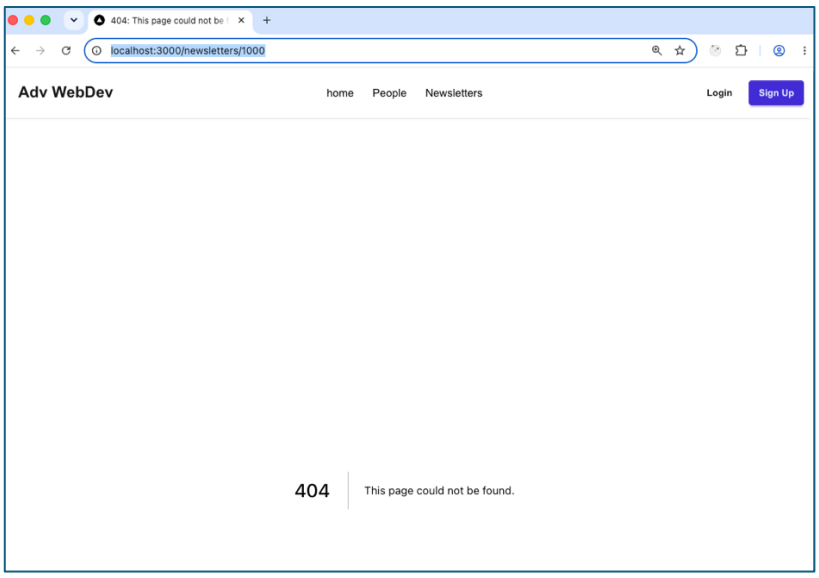
            <div className="card-actions justify-end">
              <button className="btn">Subscribe</button>
            </div>
          </div>
        </div>{" "}
      </>
    ) : (
      <div>{notFound()}</div>
    )}
  </>
);
```

The subscribe button does not work yet, as we have not implemented it.

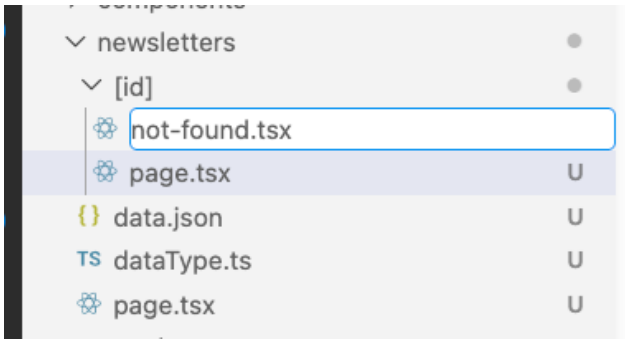
vi. Save the file and test it! My outputs for your information:



vii. Try incorrect id - http://localhost:3000/newsletters/1000 , you should see a page similar to below:



viii. Create a new file called '`not-found.tsx`' inside the [id] folder:

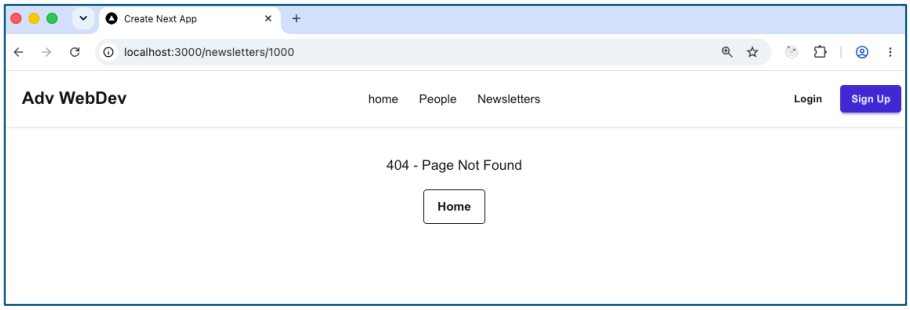


ix. Code this not found component as follow:

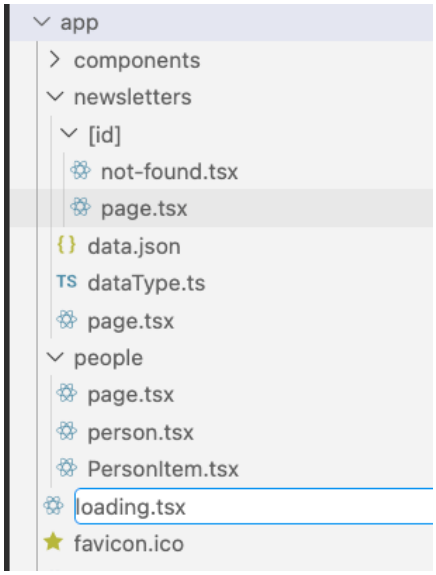
app > newsletters > [id] > not-found.tsx > ...

```tsx
import Link from "next/link";
export default function NotFound() {
  return (
    <div className="max-w-md mx-auto p-4 my-4 text-center">
      <h1>404 - Page Not Found</h1>
      <Link className="btn btn-outline mt-4" href="/">
        Home
      </Link>
    </div>
  );
}
```

x. Now refresh or try another invalid id, you should a page similar to below:



j. We will create a function for global loading. In the app folder, create a new file called "`loading.tsx`". Make sure the file name is `loading.tsx` and it is placed in 'app' folder, otherwise it won't work.



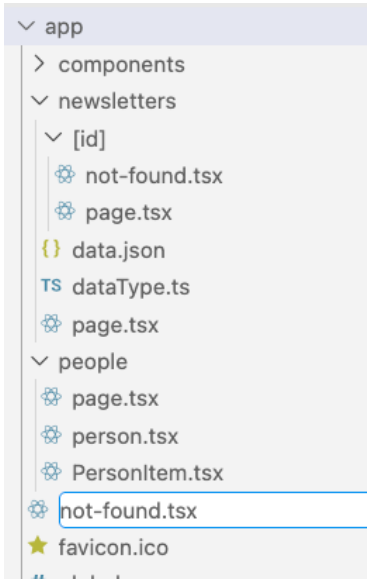
k. Open this file and code the following:

app > loading.tsx > ...

```tsx
export default function GlobalLoading() {
  return (
    <div className="min-h-[50vh] flex items-center justify-center">
      <div className="text-center">
        <span className="loading loading-spinner loading-lg text-primary"></span>
        <p className="mt-4 text-base-content/70">Loading...</p>
      </div>
    </div>
  );
}

```

l. Create another file called "`not-found.tsx`" inside the "app" folder:



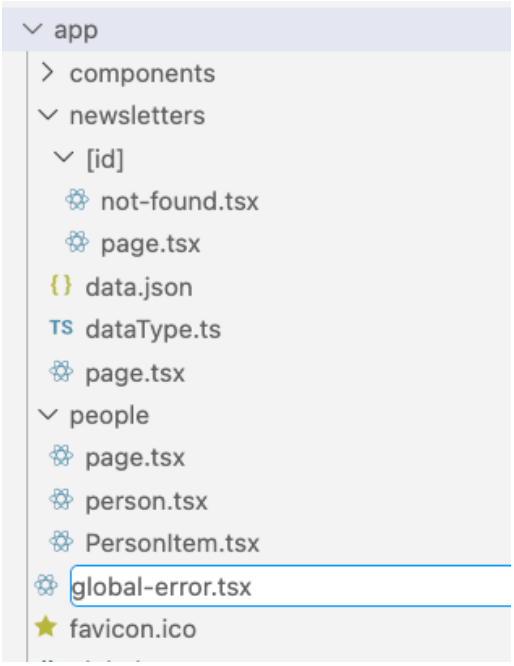
m. Open this file, and code the following:

app > not-found.tsx > ...

```tsx
import Link from "next/link";

export default function NotFound() {
  return (
    <div className="min-h-[50vh] flex items-center justify-center p-4">
      <div className="text-center">
        <div className="text-8xl font-bold text-primary mb-4">404</div>
        <h2 className="text-2xl font-bold mb-2">Page Not Found</h2>
        <p className="text-base-content/70 mb-6">
          The page you&apos;re looking for doesn&apos;t exist or has been moved.
        </p>
        <div className="flex gap-4 justify-center">
          <Link href="/" className="btn btn-primary">
            Go Home
          </Link>
          <Link href="/skills" className="btn btn-ghost">
            Browse Skills
          </Link>
        </div>
      </div>
    </div>
  );
}
```

n. Create another file called "`global-error.tsx`" inside "app" folder:



o. Open this file and code the following:

app > global-error.tsx > ...

```tsx
"use client";

export default function GlobalError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  return (
    <html>
      <body>
        <div className="min-h-[50vh] flex items-center justify-center p-4">
          <div className="text-center">
            <div className="text-6xl mb-4">😵</div>
            <h2 className="text-2xl font-bold mb-2">Something went wrong!</h2>
            <p className="text-base-content/70 mb-4">
              {error.message || "An unexpected error occurred"}
            </p>
            <div className="flex gap-4 justify-center">
              <button onClick={() => reset()} className="btn btn-primary">
                Try again
              </button>
              <a href="/" className="btn btn-ghost">
                Go home
              </a>
            </div>
          </div>
        </div>
      </body>
    </html>
  );
}

```

p. Now test it!
   i. Try : http://localhost:3000/peopleee
   ii. Try set network to 3G and test the loading
   iii. Try : http://localhost:3000/newsletters/555

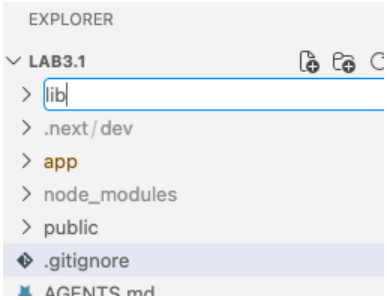
Part B: RTK implementation

7. We will now implement RTK to manage the states on client components. RTK documentation can be found : https://redux-toolkit.js.org/usage/nextjs . So do the following:
    a. Stop the development server- ^C
    b. Install RTK - `npm install @reduxjs/toolkit react-redux`

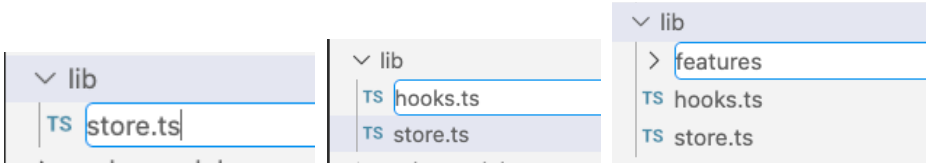


    c. Restart the development server

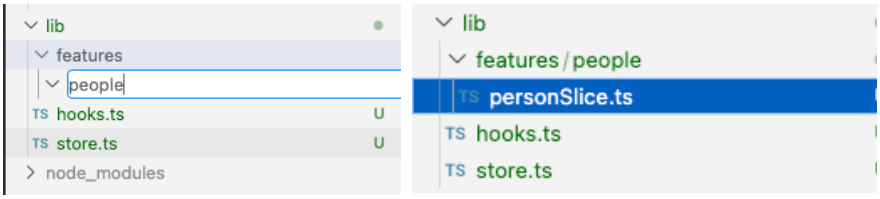
8. RTK documentation recommended that the files should be stored in a folder called '`lib`', so we will do the same. Create a new folder called '`lib`':



9. Inside this '`lib`' folder, create a new file called '`store.ts`', '`hooks.ts`', and a new folder called '`features`':



10. Inside the '`lib/features`' folder, create a subfolder called '`people`', and inside this '`people`' folder, create a file called '`personSlice.ts`' :



11. Open the `store.ts` file, import the configure store from RTK. Follow the documentation (https://redux-toolkit.js.org/usage/nextjs), and create one Store per Request (copied and pasted to the file)

lib > TS store.ts > ...

```ts
import { configureStore } from '@reduxjs/toolkit'

export const makeStore = () => {
  return configureStore({
    reducer: {},
  })
}


// Infer the type of makeStore
export type AppStore = ReturnType<typeof makeStore>
// Infer the `RootState` and `AppDispatch` types from the store itself
export type RootState = ReturnType<AppStore['getState']>
export type AppDispatch = AppStore['dispatch']


```

12. Open the `hooks.ts` and code the following:

lib > TS hooks.ts > useAppStore

```ts
import { useDispatch, useSelector, useStore } from 'react-redux'
import type { RootState, AppDispatch, AppStore } from './store'

// Use throughout your app instead of plain `useDispatch` and `useSelector`
export const useAppDispatch = useDispatch.withTypes<AppDispatch>()
export const useAppSelector = useSelector.withTypes<RootState>()
export const useAppStore = useStore.withTypes<AppStore>()
```

13. Open the `personSlice.ts`, import the `createSlice` from RTK, code an initial state, and create the `personSlice` as follow, print the slice to the console as well:

lib > features > people > TS personSlice.ts > ...

```ts
import { createSlice } from "@reduxjs/toolkit";

/// createAsyncThunk is used for async tasks

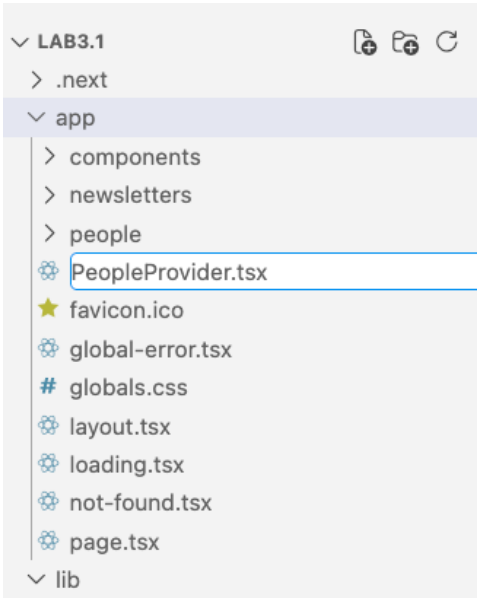
const initialState = {
    results : [],
    isLoading: true,
}

const personSlice = createSlice({
    name: "people",
    initialState,
    reducers: {}

})

console.log(personSlice);
```

14. We need a client component to access the store. In the root folder, create a new file called '`PeopleProvider.tsx`':



15. Open this file, copy and paste the code from the documentation site to this file:

app > PeopleProvider.tsx > ...

```tsx
'use client'
import { useRef } from 'react'
import { Provider } from 'react-redux'
import { makeStore, AppStore } from '../lib/store'

export default function PeopleProvider({
  children,
}: {
  children: React.ReactNode
}) {
  const storeRef = useRef<AppStore>(undefined)
  if (!storeRef.current) {
    // Create the store instance the first time this renders
    storeRef.current = makeStore()
  }

  return <Provider store={storeRef.current}>{children}</Provider>
}

```

Note: I call my provider '`PeopleProvider`'

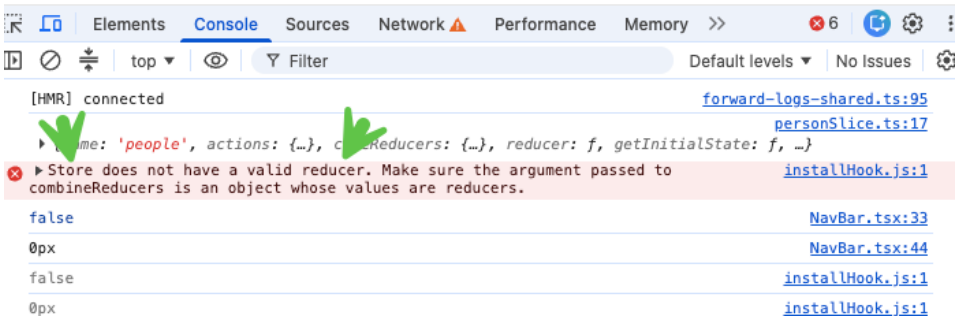
16. Next, we will wrap this provider in the root element of the app. Open the `layout.tsx`, import the provider:

```tsx
import PeopleProvider from './PeopleProvider'
```

17. Then wrap the provider as follow:

```
>
  <body className="min-h-full flex flex-col">
    <PeopleProvider>
    <Toaster position="bottom-right" />
      <NavBar />
    {children}
    </PeopleProvider>
    </body>
</html>
```

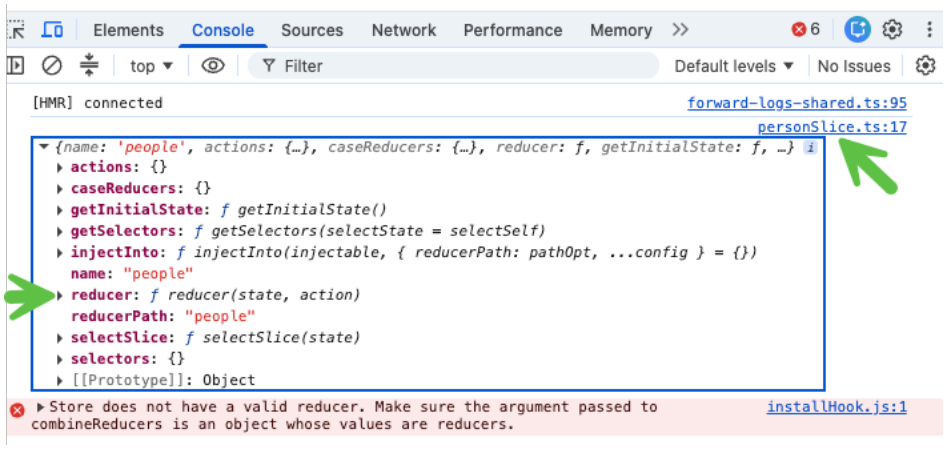
18. Save the file and inspect the console tab. At this point you will see an error on the web console, this is because we have not configured the '`store`' yet, which we will do next.



19. Now back to `store.ts`, and import the `personSlice`:

```
lib > TS store.ts > ...
1   import { configureStore } from '@reduxjs/toolkit'
2   import '@/lib/features/people/personSlice'
```

20. On the web console, examine the content of the `personSlice` object. Here, we are interested in its '`reducer`' property:



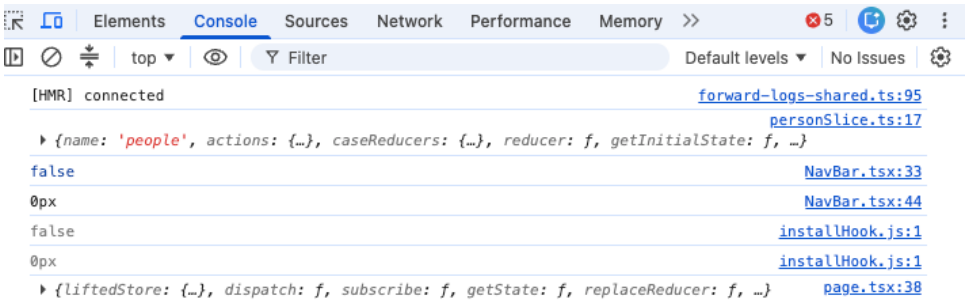
21. So, navigate back to our '`personSlice`', we want to export its 'reducer', as shown below:

```
10  const personSlice = createSlice({
11      name: "people",
12      initialState,
13      reducers: {}
14
15  })
16
17  console.log(personSlice);
18
19  export default personSlice.reducer;
```

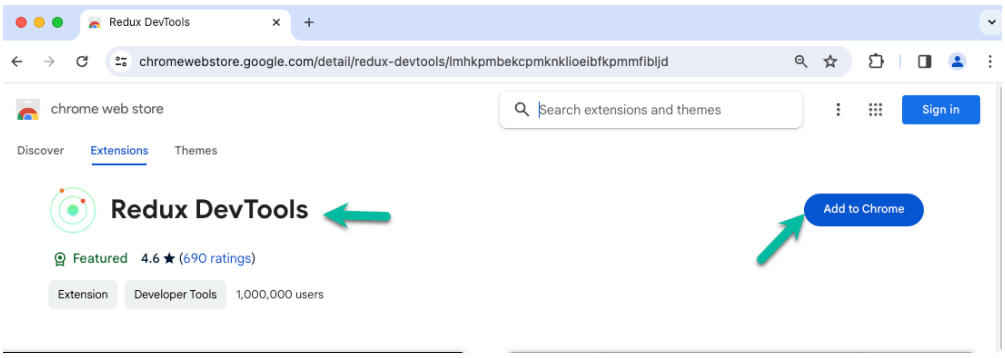
22. Once the `personSlice.reducer` is exported, we can import it in our '`store.ts`', line 3, and configure the '`store`' reducer, line 8, as shown below:

```
lib > TS store.ts > ...
1   import { configureStore } from '@reduxjs/toolkit'
2   import '@/lib/features/people/personSlice'
3   import personReducer from './features/people/personSlice'
4
5   export const makeStore = () => {
6     return configureStore({
7       reducer: {
8           people: personReducer,
9       },
10    })
11  }
12
```

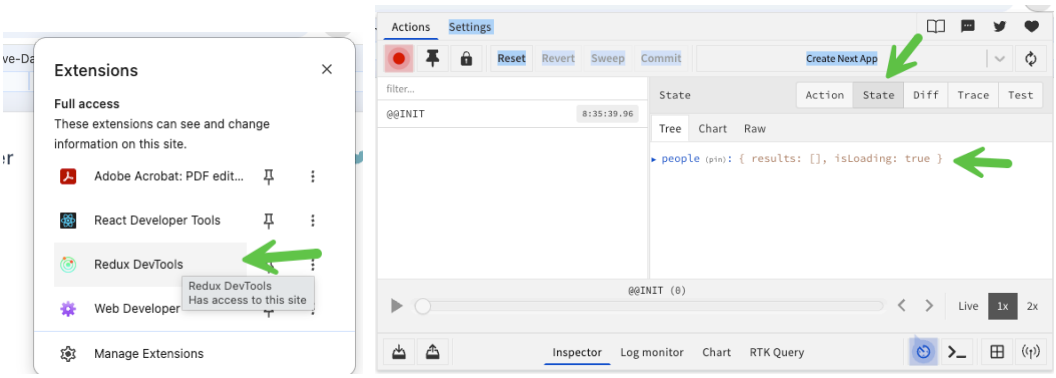
23. Save the file. Notice that the error in the web console has gone.



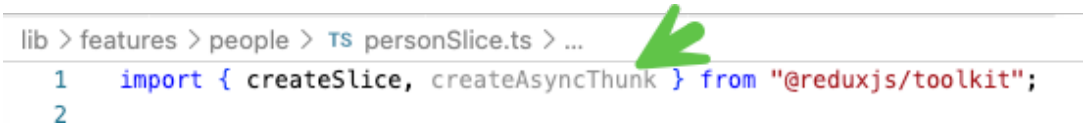
24. To see Redux action on the browser, we will install “Redux Dev Tools” extension on Google Chrome. Note: the lab machines have already been installed with this extension but if you are using your own machine, you will need to install it yourself.



25. Once the extension is installed, restart your Chrome, and you will see the ‘Redux’ tab, and our initial ‘state’ is shown as well:



26. Now, what we want to do is to put all the actions in our ‘`personSlice`’, including fetching the data. As we know, fetching external resources is asynchronous in JavaScript. Redux Toolkit provides a function called ‘`createAsyncThunk`’ for this purpose (https://redux-toolkit.js.org/api/createAsyncThunk) , it will manage the asynchronous functions for us. So, import this “`createAsyncThunk`’ in our ‘`personSlice`’, as shown below:



27. Next, we will write a function to invoke this '`createAsyncThunk`', I called this function '`fetchRandomUsers`':

```
10
11    const fetchRandomUsers = createAsyncThunk()
12
```

28. The `createAsyncThunk` takes 3 parameters – an action type, a callback function, and an option object. The action type is a string that will be used to generate additional Redux action type constants, representing the lifecycle of an async request:

```
10
11    const fetchRandomUsers = createAsyncThunk('', async () =>{
12
13    })
14
```

29. Here I called my action type '`people/getRandomUsers`'. The second argument is 'payloadCreator' callback function. This function takes 2 parameters, first one is the request id, and the second one is 'thunkAPI', an object that can be passed to a Redux 'thunk' function. So, the structure of the code looks like this, 'name' represents the request id, and the 'thunkAPI' object as the second parameter, see line 14. Export the function too.

```
13
14  export const fetchRandomUsers = createAsyncThunk('people/getRandomUsers', async (name, thunkApi) =>{
15      const response = await axios.get(url);
16      return response.data.results;
17  })
18
```

Make sure Axios is imported and the url is configured as well, otherwise it won't work.

lib > features > people > TS personSlice.ts > personSlice

```
1   import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
2   import axios from 'axios';
3
5
6   const url = "https://randomuser.me/api/?results=12"
7
```

30. We also want to implement error handling, code the async call back in a try block:

```
13
14  export const fetchRandomUsers = createAsyncThunk('people/getRandomUsers', async (name, thunkApi) =>{
15      try{
16          const response = await axios.get(url);
17          return response.data.results;
18      } catch(error) {
19          console.log(error)
20      }
21
22  })
```

31. The '`thunkAPI`' (see https://redux-toolkit.js.org/api/createAsyncThunk) has a property '`rejectWithValue`' we can use to handle the error. So here, we can simply return the error, and '`Axios`' stores the error in '`error.response`'. Because we are handling `Axios` specific error, we will need to check if the error is from Axios API. Here is final code:

```
14  export const fetchRandomUsers = createAsyncThunk('people/getRandomUsers', async (name, thunkApi) =>{
15      try{
16          const response = await axios.get(url);
17          return response.data.results;
18      } catch(error:any) {
19          if (error.isAxiosError)  {
20           console.log(error.response)
21           return thunkApi.rejectWithValue(error.response)
22          } else {
23           console.log(error);
24          }
25
26      }
27  })
```

32. Now that the '`fetchRandomUsers`' function is complete, we will move on to our '`personSlice`'. There are a couple more properties we can configure, (1) `reducers` - which takes an object of actions, and (2) '`extraReducers`' - which takes a function and a '`builder`' as its parameter:

```
30  const personSlice = createSlice({
31      name: "people",
32      initialState,
33      reducers: {},
34      extraReducers: (builder) => {},
35  })
36
```

33. We will configure the '`extraReducers`' first as we need this to handle our asynchronous fetch. This property is generally used for handling async functions. It takes an object called '`builder`' as the parameter, and in turn, we can use this '`builder`' object to handle the promise returns by Axios API. The syntax is '`builder.addCase()`'. Recall Promise has 3 states, pending, fulfilled and rejected. Here, we will handle these cases one by one, line 35 to line 47. Notice that we access the '`initialState`' by referencing to the `state` and using the '`state.`' notation (i.e. `state.isLoading`, `state.randomUsers`), and update the value directly:

```
30  const personSlice = createSlice({
31      name: "people",
32      initialState,
33      reducers: {},
34      extraReducers: (builder) => {
35          builder.addCase(fetchRandomUsers.pending, (state) =>{
36              state.isLoading = true;
37
38          });
39          builder.addCase(fetchRandomUsers.fulfilled, (state, action) =>{
40              console.log(state);
41              console.log(action.payload);
42              state.results = action.payload;
43              state.isLoading = false;
44              console.log(state.results.length);
45
46          });
47          builder.addCase(fetchRandomUsers.rejected, (state) =>{
48              state.isLoading = false;
49
50          });
51      },
52  })
53
54  console.log(personSlice);
55
56  export default personSlice.reducer;
```

34. Now, to see this in action, we will need to modify our 'People' component. The easier way to do this is to remove everything we have coded apart from <`return`> statement, and we will code this step by step, here is my code (after deleting everything between the component and <`return`> statement):

Before:

app > people > page-origin.tsx > PeoplePage

```tsx
const PeoplePage = () => {
  const [users, setUsers] = useState<Result[]>([]);

  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    axios
      .get(url)
      .then((response) => {
        console.log(response);
        setUsers(response.data.results);
        setIsLoading(false);
      })
      .catch((error: any) => {
        if (error.isAxiosError) {
          console.log(error.response);
          toast.error(`Error occurred while fetching..`);
        } else {
          console.log(error);
        }
      });
  }, []);

  if (isLoading) {
    return (
      <>
        <span className="loading loading-ball loading-xs"></span>
        <span className="loading loading-ball loading-sm"></span>
        <span className="loading loading-ball loading-md"></span>
        <span className="loading loading-ball loading-lg"></span>
        <span className="loading loading-ball loading-xl"></span>
      </>
    );
  }

  return (
    <section>
      <h2 className="font-bold text-center text-amber-900 tracking-tight mt-2 text-
        People Page
```

After:

app > people > page-origin.tsx > ...

```tsx
"use client";

import axios from "axios";
import type { Result } from "./person";
import PersonItem from "./PersonItem";
import { useEffect, useState } from "react";
import { toast } from "react-hot-toast";

const url = "https://randomuser.me/api/?results=12";

const PeoplePage = () => {

  return (
    <section>
      <h2 className="font-bold text-center text-amber-900 tracking-tight mt-2 text-3xl">
        People Page
      </h2>
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 p-6">
        {users?.map((user) => {
          return (
            <PersonItem {...user} key={user.login.uuid} remove={() => {}} />
          );
        })}
      </div>
    </section>
  );
};

export default PeoplePage;

```

35. After the clean-up, do the following: First, import '`useAppDispatch`', '`useAppStore`' and '`useAppSelector`' from the `hooks.ts`, see line 8:

```
7
8   import { useAppSelector, useAppDispatch, useAppStore } from '../../lib/hooks'
9
```

36. Then, inside the 'People' component, we use '`useAppSelector`' to reference the state values:

```
16    const PeoplePage = () => {
17      const { results, isLoading } = useAppSelector((store) => store.people);
18
```

You may be wondering where is this '`store`' come from? It is the prop passed down from Redux 'Provider', in our case it is `PeopleProvider`, which we wrap in our '`layout.tsx`' and in our '`store`' we have a property called '`people`', which has our '`personSlice.reducer`' as its value.

37. Next, we want to invoke the '`useAppDispatch`' function, and create an object called '`dispatch`' (line 19). We use this function to help us fetching the data, and because we only want to fetch the data once at component mount time, we will call this inside the '`useEffect`' hook (line 21to line 23):

```
14  import { useAppSelector, useAppDispatch, useAppStore } from "../../lib/hooks";
15
16  const PeoplePage = () => {
17    const { results, isLoading } = useAppSelector((store) => store.people);
18
19    const dispatch = useAppDispatch();
20
21    useEffect(() => {
22      dispatch(fetchRandomUsers());
23    }, []);
24
```

Obviously the '`fetchRandomUsers`' function needs to be imported:

```
7   import { fetchRandomUsers } from '@/lib/features/people/personSlice';
```

38. Code a condition block to handle '`isLoading`', just like before, making sure that this is placed **after** the '`useEffect`' hook:

```
37    if (isLoading) {
38      return (
39        <div className="p-6 max-w-md mx-auto my-4">
40          {" "}
41          <span className="loading loading-spinner loading-sm"></span>
42        </div>
43      );
44    }
```

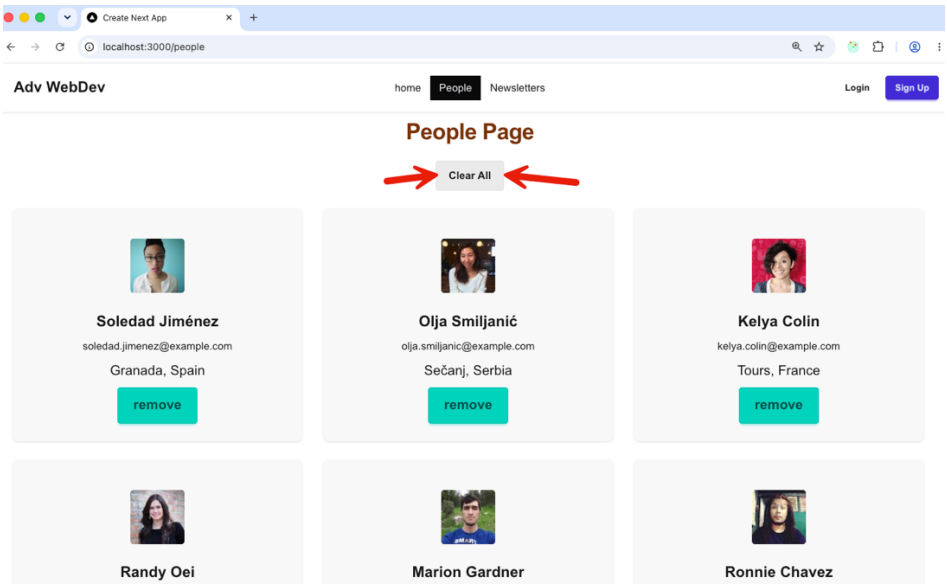
39. Now finish off the displaying results, as follow:

```
58        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 p-6">
59          {results?.map((p: Result) => {
60            return <PersonItem {...p} key={p.login.uuid} remove={() => {}} />;
61          })}
62        </div>
```

Notice the state here is "results" and not users!

40. Save the file and refresh the page. You should see the output on the page! Test the `isLoading` to see if it works as well!

41. Once you have completed the tasks above, your next task is to create a '`clear all`' button. This button should appear on the page when there are users on the list (i.e. the state value - results). Here is my output:



42. Next, we will now code the function to support this '`clear all`' action. Navigate to our '`personSlice`' and configure the '`reducers`' action: add '`clearList`' action, and reset the list to an empty array, line 34 to line 37

```
29
30  const personSlice = createSlice({
31      name: "people",
32      initialState,
33      reducers: {
34          clearList: (state) =>{
35              state.results = [];
36
37          },
38      },
```

43. Export this action, line 61:

```
60
61  export const {clearList} = personSlice.actions;
62
63  export default personSlice.reducer;
```

44. After that, navigate to '`page.tsx`' in People folder, and import this action (line 8):

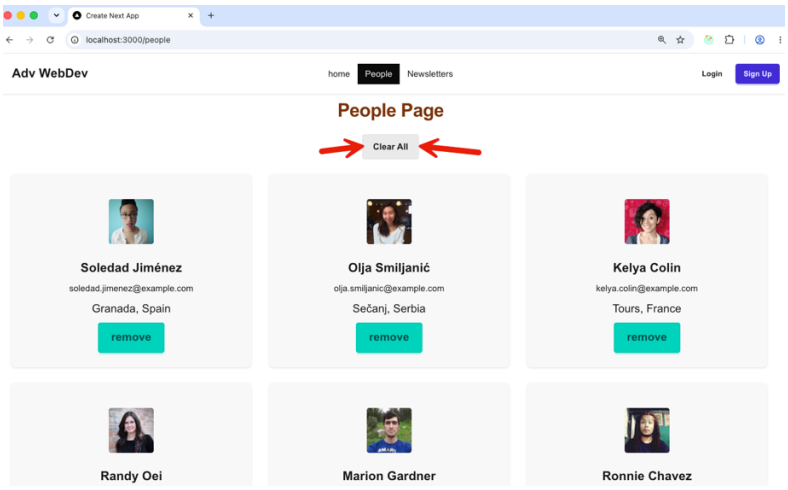
```
7
8   import { fetchRandomUsers , clearList} from '@/lib/features/people/personSlice';
9
```

45. To invoke this action, code a handler function (I called mine `handleClearList`), and use the '`dispatch`' function to invoke this action – from line 24 to line 26:

```
23
24    const handleClearList = () => {
25      dispatch(clearList());
26    };
27
```

46. The next thing is to attach this handler to the '`clear all`' button to handle the `onClick` event! Save the file and test it out!

47. Once you got it working, code a second action '`reset`', such that, a button called '`reset`' appears when there is no user on the list. When this button is clicked, a `fetchRandomUser` call is made to retrieve a new set of fake random users and display on the page. Below are my outputs (before, after) for your reference:





48. Once you have completed the tasks above, we will move on to code the remaining functions such that the users can be removed one by one. Navigate to our '`personSlice`' and configure an action called '`removeUser`' in '`reducers`' property. Here, it takes 2 arguments, '`state`' and '`action`'. For deletion we will need to keep track of the `id` so that we know which item to delete. We can access this '`id`' from the '`payload`'. (line 39 to line 43)

```
34      reducers: {
35          clearList: (state) =>{
36              state.results = [];
37
38          },
39          removeUser: (state, action) => {
40          const uuid = action.payload;
41          state.results = state.results.filter(
42              (user:Result) => user.login.uuid !== uuid );
43          },
44      },
```

Notice that user is of type `Result`. Make sure that this type is imported as well:

```
3   import { Result } from "@/app/people/person";
4
```

49. Now export this action:

```
61
62  export const { clearList, removeUser } = personSlice.actions;
63
64  export default personSlice.reducer;
65
```

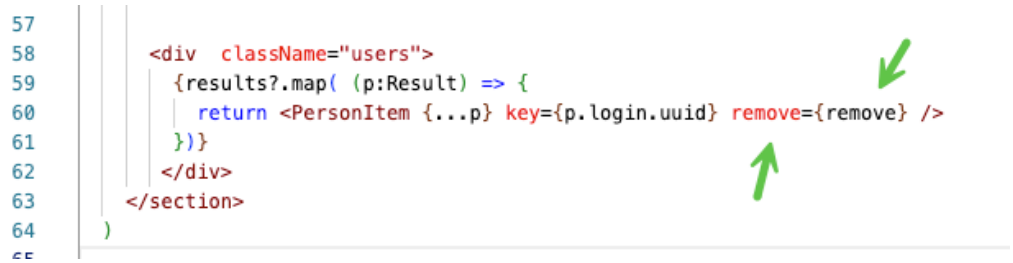
50. and then import it in 'People' component (i.e. `page.tsx` in People folder)

```
 8  import {
 9    fetchRandomUsers,
10    clearList,
11    removeUser,
12  } from "@/lib/features/people/personSlice";
13
```

51. In '`page.tsx`' People component, create a handler so that this action is invoked using a '`dispatch`' function (refer to step 40). This handler takes a parameter, which is the `id`, here is my code for your reference:

```
35  const remove = (id:any) => {
36    dispatch(removeUser(id));
37  };
```

52. Pass this handler as `prop` to the child node (i.e. `PersonItem`):



53. Save the file and test it!

The final task to commit the code to CMP Stugit and name your repo as "<your-username>lab3.1"